# `trainer.py`

Model factory, the feature-set dispatch registry, and the single-model training CLI

---

The central registry tying every feature set defined in `feature_engineering.py` to a concrete, trainable sklearn `Pipeline`, plus the command-line entry point (`python trainer.py --model ... --feature-set ...`) that trains one model, evaluates it on the held-out validation split, and persists both the metrics and the fitted pipeline. The seven feature sets it dispatches between:

| feature_set | Linear/base features | Induced features | compatible --model |
|---|---|---|---|
| `freq_agg` | context one-hot (capped 50) + smoothed target-encoded user/ad aggregates + hour | — | logreg, hist_gbdt, lightgbm |
| `baseline_ohe` | every raw categorical column, one-hot, uncapped | — | logreg, lightgbm |
| `gbdt_leaves` | — | GBDT leaves (GBDT trained on native-categorical raw columns) | logreg, lightgbm |
| `gbdt_leaves_ohe` | — | GBDT leaves (GBDT trained on uncapped one-hot) | logreg, lightgbm |
| `gbdt_leaves_concat` | uncapped one-hot | GBDT leaves (same GBDT as gbdt_leaves_ohe) | logreg, lightgbm |
| `freq_agg_leaves_concat` | freq_agg features | GBDT leaves (GBDT trained on uncapped one-hot, not freq_agg) | logreg, lightgbm |
| `freq_agg_fm_concat` | freq_agg features | FM latent projection (FM trained on uncapped one-hot, not freq_agg) | logreg, hist_gbdt, lightgbm |

## `get_model` FUNCTION

```
get_model(model_name, **kwargs) -> sklearn/lightgbm estimator
```

A simple factory: `"logreg"` → `LogisticRegression(max_iter=10000, class_weight="balanced")` (the class-weighting reweights the loss to counteract Avazu's ~16–17% base click rate); `"hist_gbdt"` → scikit-learn's `HistGradientBoostingClassifier`; `"lightgbm"` → `lightgbm.LGBMClassifier`, lazily imported with a friendly `ImportError` if the optional dependency isn't installed. Any other name raises `ValueError`.

### `train_model, save_model, load_model` FUNCTION

```
train_model(model, X_train, y_train) / save_model(artifact, path) / load_model(path)
```

Thin wrappers: `train_model` just calls `.fit()`. `save_model` / `load_model` persist/restore an *artifact* dict via `joblib` — not just the fitted pipeline, but also (for the `freq_agg` family) the `fit_stats` / `global_ctr` / `group_cols` needed to reproduce the target-encoding step on fresh raw data without the original training DataFrame around.

### `build_features` FUNCTION

```
build_features(train_df, val_df, group_cols=GROUP_COLS) -> (train_df, val_df, cat_cols,
num_cols, feature_cols, fit_stats, global_ctr)
```

Engineers the `freq_agg` feature set end to end: derives hour features, computes `global_ctr` from the training split, then delegates the leakage-safe smoothed/leave-one-out target encoding to `feature_engineering.add_freq_agg_features` (see that module's documentation for the encoding formulas). `num_cols` collects the resulting `{col}_ctr` / `{col}_count` columns for every column in `group_cols` plus the two hour-derived columns; `cat_cols` is just `CONTEXT_FEATURE_COLS` (the columns that stay one-hot rather than being target-encoded).

### `build_baseline_ohe_features, build_gbdt_leaf_features` FUNCTION

```
build_baseline_ohe_features(train_df, val_df) / build_gbdt_leaf_features(train_df,
val_df)
```

`build_baseline_ohe_features` derives hour features and returns `ALL_CAT_FEATURE_COLS + HOUR_DERIVED_COLS` as the (identical) `cat_cols` / `feature_cols` for the `baseline_ohe` feature set — no target-encoding or other transformation happens here, that's entirely delegated to the preprocessor built by `feature_engineering.build_ohe_only_pipeline`.

`build_gbdt_leaf_features` does the same for the `gbdt_leaves` feature set, but additionally calls `feature_engineering.to_lgbm_categoricals` to fix each categorical column's dtype/vocabulary from the training split, since this feature set relies on lightgbm's *native* categorical-split handling rather than one-hot encoding.

## `FEATURE_SET_COMPATIBLE_MODELS`, `check_feature_set_model_compatible` CONST+FUNCTION

```
check_feature_set_model_compatible(feature_set, model_name) -> None
```

A lookup table of which `--model` values are valid for each `--feature-set`, checked once near the start of `main()` (and, in `run_pipeline.py`, once per configured run) so an invalid combination fails fast with a clear `ValueError` rather than a confusing exception deep inside a model's `.fit()` call.

The rule driving the table: `HistGradientBoostingClassifier` requires dense input and raises on sparse *X*. Every feature set except `freq_agg` and `freq_agg_fm_concat` produces sparse output somewhere in its `FeatureUnion` / `ColumnTransformer` — including `freq_agg_leaves_concat`, whose GBDT-leaf branch really is a sparse one-hot matrix. `freq_agg_fm_concat` looks structurally identical (also a `FeatureUnion` with a GBDT/FM-leaf-shaped branch) but is *not* excluded: `FMEmbeddingEncoder.transform` returns `X @ v_`, and sparse-matrix-times-dense-array always produces a dense `ndarray` in scipy, so the whole feature set's output ends up fully dense despite its sparse internal input.

## `feature_set_engineering_key` FUNCTION

```
feature_set_engineering_key(feature_set) -> str
```

Maps several feature sets that resolve to *byte-identical* `engineer_features` output onto the same cache key, so a caller that engineers features for multiple feature sets in one run ( `run_pipeline.py` ) doesn't redundantly redo the same expensive, groupby-heavy DataFrame work.
`baseline_ohe` / `gbdt_leaves_ohe` / `gbdt_leaves_concat` all share `"baseline_ohe_family"` (all three call `build_baseline_ohe_features` with identical arguments);
`freq_agg` / `freq_agg_leaves_concat` / `freq_agg_fm_concat` all share `"freq_agg_family"` for the analogous reason. Deliberately co-located with `engineer_features` immediately below so the two definitions can't silently drift out of sync.

## `engineer_features` FUNCTION

```
engineer_features(feature_set, train_df, val_df) -> (train_df, val_df, feature_cols, state)
```

The feature-set dispatcher for the (seed-independent, potentially expensive) DataFrame engineering step — the single `if` / `elif` chain that every `build_*_features` helper above is invoked from. `state` is an opaque dict of whatever feature-set-specific data `build_preprocessor` (below) and the saved-model artifact need downstream (column lists, and for the `freq_agg` family, `fit_stats` / `global_ctr`).

Deliberately split out from `build_preprocessor` so a caller that trains multiple models on the same feature set (`run_pipeline.py`) can engineer features exactly once and build a fresh, unfit preprocessor per model afterward.

For the `freq_agg` family specifically, `feature_cols` / `state` are always computed at their *widest* — including the raw one-hot-style columns only `freq_agg_leaves_concat` / `freq_agg_fm_concat` actually need — regardless of which of the three feature sets was requested. This is safe (a downstream `ColumnTransformer` simply ignores any column it doesn't select by name) and is exactly what makes all three feature sets' output byte-identical, which is what lets `feature_set_engineering_key` group them for caching.

## `build_preprocessor` FUNCTION

```
build_preprocessor(feature_set, state, seed) -> sklearn transformer
```

The second half of the dispatcher: given the `state` produced by `engineer_features`, constructs a *fresh, unfit* preprocessing transformer for `feature_set` — one call per feature set into the matching `feature_engineering.build_*_pipeline` function. Always builds a new instance rather than reusing one, so that training multiple models on the same engineered DataFrames doesn't have them share a preprocessor object mutated in place by a prior model's `Pipeline.fit`.

### `main (CLI entry point)` FUNCTION

```
python trainer.py --model {logreg,hist_gbdt,lightgbm} --feature-set {...} [--n-rows] [--val-frac] [--seed] [--data-path] [--output-dir]
```

End-to-end single-run pipeline: validate the `--model` / `--feature-set` combination, load the configured sample, time-split it, engineer features and build the preprocessor for the requested feature set, wrap `Pipeline([("preprocess", preprocessor), ("model", model)])`, fit on train, evaluate on validation via `evaluator.evaluate`, print the metrics, then persist both a metrics JSON and a joblib model artifact.

Output filenames follow one convention throughout the project: `freq_agg` (the original, default feature set) keeps unsuffixed filenames ( `{model}.joblib` / `metrics_{model}.json` ) for backward compatibility with earlier results; every other feature set gets a disambiguating suffix ( `{model}_{feature_set}.joblib` / `metrics_{model}_{feature_set}.json` ).